Continuous Normalising Flow-based Generative Adversarial Networks and Introductory Differential Equation Theory

Darshan Hindocha

Master of Science in Data Science
The University of Bath
2021

Continuous Normalising Flow-based Generative Adversarial Networks and Introductory Differential Equation Theory

Submitted by: Darshan Hindocha

Copyright

Declaration

This dissertation is submitted to the University of Bath in accordance with the requirements of the degree of Bachelor of Science in the Department of Computer Science. No portion of the work in this dissertation has been submitted in support of an application for any other degree or qualification of this or any other university or institution of learning. Except where specifically acknowledged, it is the work of the author.

Abstract

The various types of generative models seek to understand the underlying distribution of some pool of data. Image data is high dimensional and complex so we cannot represent the underlying distribution using density functions. GANs focus on sampling points with high likelihoods in the unknown underlying distribution. Flow-based models use the change of variables to form a transformation from a simple distribution to the underlying distribution enabling more capabilities but poorer sampling – due to restricted architectures – and high computational costs. We combine the continuous normalising flow and GAN much like flow-GANs but with the added benefits of Neural ODEs. Furthermore, we introduce, motivate and describe the intuition behind differential equations. We believe an intuition driven discussion of differential equations can help a broader data science community better appreciate the nuances of Neural ODEs.

Contents

List of Figures

Acknowledgements

Thank you to Maharaj and Swami

Thank you to my family for the constant support. My mum for listening to my explanations despite them not being relevant to you. My dad for pushing me. My sister for constantly providing healthy competition.

Thank you to my project supervisor, Dr. Vinay Namboodiri, for providing inspiration and guidance in our weekly virtual meet-ups.

Thank you to Neel, Tilly, Torem and Max for being amazing study company through the course of the project.

Thank you to the Computer Science department for letting me use Hex – the friendly neighbourhood GPU cloud.

Thank you to Starbucks and Pret for the strong coffee.

Chapter 1

Introduction and Literature Review

1.1 Background

In the past few decades, the significant advance of computing efficiency, accumulation of data and development of machine learning theory has led to the rapid and widespread adoption of deep learning. A catalyst for developing a field of study is in finding and tapping into the connections with other well studied subjects. For example, some important developments within deep learning were inspired by neuroscience (*e.g.*, Glorot, Bordes and Bengio, 2011). An exciting connection was made recently in Chen et al. (2018), connecting deep learning and differential equations in a novel way.

Deep learning and differential equations have overlapping goals. We describe why both fields are well suited for one another in a later section. The trouble is that differential equations do not appear in significant capacity in computer science. Therefore, data scientists coming from a computer science background may not have much experience working with differential equations. On the other hand, mathematicians study differential equations in detail but mostly in an overly abstract form. Data scientists coming from a pure mathematics background may not have much experience in the application of differential equations.

In this thesis we provide motivation and intuition regarding the application of differential equations. We focus on the concepts relevant to understanding the Neural Ordinary Differential Equations posed in Chen et al. (2018). The reasons for using differential equations (DEs), the different categories of DEs, the way to understand DEs, the way to solve DEs is all vastly different to other topics in the space of function approximation. The simple description of using a neural network as a differential equation can have a significantly different meaning for those comfortable with DEs versus those who are not.

With our discussions of differential equations in this thesis we hope to provide visually interpretable explanations, motivation for why and how to solve and understand differential equations, and an appreciation for the challenges and limitations of differential equations. Given the breadth of what we wish to cover, the topics are explained in less detail and with intuitive descriptions.

The significant contributory focus of this work is on further developing the connection between differential equations and deep learning through a proposed continuous normalising flow-based generative adversarial network. A generative model capable of efficiently sampling points and tractably calculating likelihoods of samples presents highly useful advantages (Kingma and Dhariwal (2018)). The ability to tractably calculate likelihoods comes at the cost of model flexibility and sample quality. The continuous normalising flow – arising from neural ordinary differential equations Chen et al. (2018) – suggests a potentially more flexible, and computationally cheaper counterpart to the family of normalising flow models. We develop the CNF-GAN model for generative modelling tasks and conduct experiments investigating the effectiveness of the model in a basic generative task.

1.2 Literature and Technology Survey

Our literature and technology survey gathers the research that is relevant to our thesis. Mostly this includes generative adversarial networks and neural ordinary differential equations. We simply present, briefly describe, and attribute the relevant research here without detail explanations. We save the explanations for the main content of the thesis.

1.2.1 Generative Adversarial Networks Literature

Generative Models

Generative models have experienced a surge of interest within the last decade. Ruthotto and Haber (2021) investigates and presents 'deep generative models' - a category of generative models that have neural architectures. As described in the paper, the goal of a generative model is to approximate the data sets underlying probability distribution accurately. Doing so enables the generative model to compute the likelihood that some data is from the underlying distribution and generate data points that have high likelihoods w.r.t. that distribution.

We can mathematically formalise generative modelling as finding some function $g : \mathcal{Z} \to \mathcal{X}$ where $\mathcal{Z} \in \mathbb{R}^m$ is some latent distribution and $\mathcal{X} \in \mathbb{R}^n$ is some unknown target (or data) distribution. The dimensionality of the target space corresponds to the shape of the data that we wish to study.

Introducing Generative Adversarial Networks

Generative adversarial networks (GANs) are a class of deep generative models known for their ability to reproduce extremely realistic visual samples. They were introduced in Goodfellow et al. (2014). GANs (typically) have two neural network components, the generator and the discriminator.

Researchers have developed the theory behind GANs consistently since their introduction in 2014. We will touch on some of these papers and their contributions to GANs as it lays the foundation for the open research questions.

In a later section, we discuss contributions to GANs that specifically targeted the problems faced by GANs. For now, we introduce and discuss pivotal GAN-based papers. Certain papers focus on the generator network, others on the discriminator networks, some on loss functions and others on the optimisation objective. We focus on visual computing related papers.

InfoGAN The Information Maximising GAN is introduced in Chen et al. (2016). The original GAN does not control the choice of *mode* values for which the model generates samples. Chen et al. (2016) proposes an architecture that enables the generator's input to be linked with the generated data's mode values, thereby giving control to specific properties in the data generated. This remains an unsupervised approach as the modal categories are found using unsupervised methods.

Semi-Supervised GAN Generative modelling is not limited to unsupervised data. Odena (2016) proposes a semi-supervised GAN. That is, the discriminator classifies input data into (k+1) classes where there are k classes in the original data set. The extra class for the discriminator is the class for 'fake' data that the discriminator predicts is synthetic. Odena (2016) show that the semi-supervised GAN generates high quality samples with fewer labelled training data. A semi-supervised model such as this one should be separated from the unsupervised GAN when considering performance as the extra information provides a significant advantage and is not always available.

FlowGAN The Flow-GAN proposed in Grover, Dhar and Ermon (2017) is highly relevant to this thesis. It is generally challenging to evaluate generative models because we are trying to model an unknown complicated data distribution. Furthermore, the likelihood of samples is intractable to calculate so we instead turn to how realistic the generated data looks and certain other metrics.

Flow-GAN proposes a normalising flow-based generator in a GAN. If the flow model has the required invertibility properties and certain conditions hold true then these generators can obtain their log-likelihoods feasibly. This allowed the authors to compare the log-likelihoods of the generated samples alongside the typical evaluation of how realistic the samples are. Interestingly, the paper found that the realistic samples from Flow-GAN trained adversarially had poor log-likelihoods.

ODE-GAN Qin et al. (2020) proposes the ODE-GAN to mitigate stability issues in training GANs. The premise for the architecture is based on the hypothesis that the instability is a consequence of an integration error resulting from the discretisation of the training dynamics. The paper connects training GANs to solving ODEs and they propose the ODE-GAN algorithm.

Style-GAN Karras, Laine and Aila (2018) propose the style-GAN which infuses latent information at different stages of the generator. The simple latent information trick significantly improved sample quality.

BigGAN Brock, Donahue and Simonyan (2018) propose the BigGAN. As the name suggests, they develop GANs with parameter counts reaching 100 million. The BigGANs are able to generate high fidelity synthetic samples on data with broad modality.

U-Net GAN Schönfeld, Schiele and Khoreva (2020) recently published this paper proposing a GAN with a U-net discriminator – built on top of the BigGAN. The U-net discriminator is a type of convolutional architecture proposed in Ronneberger, Fischer and Brox (2015).

Limitations and Open Research Questions

The main limitations faced by GANs are listed here. Extensive details of the issues are not explored. The focus is on discussing the relevant literature that addresses these limitations, leading to the open research questions surrounding GANs.

The Collapse of Mode The distributions of real-world data are typically multi-modal. That is, we can classify the data into modes. For instance, a data set of images of human faces will include faces with different eye colours. The issue of mode collapse (Hong (2019)) is when the generator only produces data for a certain subset of all modes in the data set. The equivalent of only producing faces with brown eyes when other eye colours are present in the data set, from the previous example.

Convergence Issues Mode collapse is a form of non-convergence. Another non-convergence is oscillation. This is where the generator continues to generate samples in training that retain modal variety but do not improve quality as training continues.

Goodfellow et al. (2014) describe the non-convergence issue in further detail is as follows (Goodfellow et al. (2014)). Notably, the optimal point for the objective function in a mini-max game in game theory is the Nash equilibrium (Nash (1950)). The Nash equilibrium for a 2-player mini-max game is defined as the point at which a player does not change its strategy regardless of the adversary's strategy. The objective optimisation target for the GAN in training is to reach the Nash equilibrium.

Many research papers in the last decade are motivated by this problem. Some papers suggest variations of GANs specifically for this problem – such as the WGAN (Arjovsky, Chintala and Bottou (2017)). Other research speculates on when and why GANs experience non-convergence problems. For instance, the cost function for GANs are notoriously precarious and Goodfellow et al. (2014) considers some issues surrounding the design of cost functions and proposes a workaround.

Evaluation Ambiguity The evaluation process for GANs is in itself a research question. With generative models, and unsupervised learning more broadly, we do not have access to the typical classification accuracy metrics. Furthermore, the log-likelihoods in generative models are intractable to calculate. Turning to how realistic a generated sample is, may not inform us of non-convergence issues (Theis, van den Oord and Bethge (2016)). Evaluation metrics are an important consideration for this work as we look to compare different GAN architectures.

Borji (2018) provides a breakdown of various qualitative and quantitative evaluation tools, including detailed descriptions, references to source papers and discussions of each one. Other resources in this area that provide guidance include Lucic et al. (2018) and Salimans et al. (2016).

1.2.2 Neural Ordinary Differential Equations Literature

Differential Equations

We discuss differential equations in depth in this thesis (see Chapter 2). For the most part of our discussion of differential equations we are simply restating and motivating well established differential equation theory. Igorevic and Silverman (1998) is high quality material for the differential equations topics we introduce, and provides thorough detail.

Neural Ordinary Differential Equations

Chen et al. (2018) introduces the concept of using a neural network in the ordinary differential equation. A neural ordinary differential equation is one way to integrate differential equations into the structures of neural networks. We describe neural ordinary differentials in depth later in this thesis.

We consider the benefits, limitations and current challenges surrounding neural ordinary differential equations. The neural ordinary differential equation is an implicit function approximation (see Section 3.1.4). This means we cannot use backpropogation to train a neural ordinary differential equation (when it is defined implicitly). Chen et al. (2018) proposes using the adjoint sensitivity method as a work around for training implicitly defined neural ordinary differential equations.

Neural ordinary differential equations were famously inspired by ResNets from He et al. (2016). Chen et al. (2018) shows that ResNets are discretised versions of implicitly defined neural ordinary differential equations. We discuss this connection in Section 3.1.5.

Lu et al. (2017) explores the general connection between deep learning and differential equations. They argue that certain neural networks are discretised, and explicitly defined versions of differential equations.

FFJORD The Jacobian Matrix is an important part of reversible generative models because they use the Change of Variables Formula (CVF) – which requires computations of the determinant of the Jacobian. For high dimensional generative tasks this computation is prohibitive. The continuous normalising flow – proposed in Chen et al. (2018) – derives a continuous analogue of the CVF that only requires the trace of the Jacobian be computed. This is generally computationally cheaper. Grathwohl et al. (2018) further reduce the computational cost of the CVF by proposing an unbiased estimator for the trace of Jacobian that has approximately squared cost on the data dimension – as opposed to cubic cost on the data dimension.

RNODE: Jacobian and Kinetic Regularisation Finlay et al. (2020a) propose regularisation terms on the loss functions used for training neural ordinary differential equations that promote simpler dynamics in ODENets (see Section 3.2.1 for ODENets). The regularisation term is derived in the paper and is inspired by differential equation theory and optimal transport.

Augmented Neural Ordinary Differential Equations Dupont, Doucet and Teh (2019) investigates the modelling ability of neural ordinary differential equations. They notice that the characteristic that neural ordinary differential equations have of preserving the topology of the input space through its dynamics, result in specific functions it theoretically cannot express. They propose the augmented neural ordinary differential equation in light of this.

Faster ODE Adjoints via Semi-Norms Kidger, Chen and Lyons (2020) investigates the error calculation used in numerical methods for neural ordinary differential equations. They propose a looser requirement for rejecting a step in the numerical solver based on the composition of augmented dynamics in the adjoint sensitivity method. The looser requirement results in fewer computations and does not reduce the accuracy of the numerical solver or model.

Advantages of Neural ODEs

By their construction, neural ordinary differential equations are continuous-depth models. Ayyubi, Yao and Divakaran (2020) investigates leveraging this for use in irregularly sampled time series data. Rubanova, Chen and Duvenaud (2019) explain the lack of sophisticated solutions in deep learning for this problem.

The other advantage of the neural ordinary differential equations comes from the continuous analogue of change of variables formula, which enable neural ordinary differential equations to be used for generative modelling. This is advantage is the focal point of the experiments in this thesis.

Further to this, neural ordinary differential equations present other characteristics that may be advantageous. For instance, they have an adaptive time property (Chen et al. (2018)) based on the tolerances of the numerical solvers we use.

Chen et al. (2018) very briefly mention in the density estimation experiment that the learning of the continuous normalising flow seems more intuitive and sensical than the learning by a normalising flow. We found this very interesting and possible advantage rooted in the continuity and other such properties of the dynamics of neural ordinary differential equations.

Limitations and Open Research Questions

As touched on previously, the paper Dupont, Doucet and Teh (2019), sheds light on transformations that neural ordinary differential equations theoretically cannot represent. Variations on the dynamics of neural ordinary differential equations is open, and while there are common dynamics, this is an area with potential.

An obstacle to the application of neural ordinary differential equations in practice for high dimensional problems is that these models remain slow to train in comparison to standard neural networks. Typically this issue is caused by highly erratic dynamics that require many more function evaluations. Finlay et al. (2020a) propose regulariser terms to promote simpler dynamics. Numerically solving complicated dynamics is a big limitation.

Software for implementing NODEs

There is some choice for what software we can use to build our models. A good numerical solver suite with GPU support is hugely advantageous. We ultimately chose to use TorchDiffEq and Python's Pytorch library for our models.

1.3 Project Objectives

There are two main objectives for this thesis.

1. **Exploring Differential Equations:** We wish to explore via discussion the theory behind NODEs and differential equations.
2. **Create and Test the CNF-GAN:** We hope to prove by example that a continuous normalising flow-based generative adversarial network (CNF-GAN) could successfully train on the MNIST data set in an unsupervised, generative capacity. If successful we wish to run experiments comparing the CNF-GAN against normalising flows, stand-alone

continuous normalising flows and GANs in terms of sample quality and latent space interpolation.

We hope to present NODEs in digestable form for those who are not comfortable with differential equations. We believe that understanding NODEs will be beneficial for machine learning engineers as through the years limitations of using such models are gradually overcome.

The flow of this work is as follows. We first introduce differential equations from an intuitive perspective. We focus on the aspects of differential equations that are particularly relevant for neural differential equations. We then pivot to NODEs and how they operate using visual examples and by drawing comparisons to common models. We introduce CNFs and discuss their architecture and the concept of reversibility. From there we move to our experiments - discussing methodology, results, analysis and providing a self-evaluation.

Chapter 2

Differential Equations

2.1 Motivating Differential Equations

2.1.1 Introduction

The purpose of this chapter is to introduce the relevant parts of differential equations in a way that helps us better understand NODEs.

The theory of differential equations is vast and deep. While the tools that differential equations provide are powerful, they come with a necessary complexity that can make it hard to grasp what exactly is going on without formal training.

In this section we hope to provide some intuition surrounding differential equations and their connection with neural networks. The intuition will hopefully serve data scientists either as refresher material for those that studied differential equations in a previous life, or as introductory material for newcomers to differential equations.

The connection between differential equations and neural networks in the way Chen et al. (2018) describes is new. Many papers and articles have been published since then, extending the ideas. However, many of these works, equipped with a thorough academic background in differential equations, skim through details that are not obvious to individuals that have not worked with differential equation.

Given the lack of differential equations appearing in deep learning, much of the ML community is at a loss and may struggle with understanding and working with NODEs and similar intersections of deep learning and differential equations.

Differential Equations are useful tools when it is possible to describe change. Differential equations have found homes in many scientific fields and applications. As a result, many abstract components of differential equations inherent parts of the applications they serve. For instance, in the vast majority of applications, the derivatives in differential equations are with respect to time. As we will see later, the differential equations in NODEs are a rare case where we are interested in derivatives with respect to something other than time - network depth. These implicit conventions can be highly confusing and only lightly discussed so we hope to address them with appropriate detail.

2.1.2 How Many Rabbits and Foxes?

We dedicate this section to motivating differential equations via example.

Using naive function approximation to find the trajectory of parameters over an independent variable in a system: Consider a set $S_{R,F} = \{R_{Pop,t}, F_{Pop,t}\}$ where the elements are variables representing the regional population at time t of rabbits and foxes respectively. We might be tempted to approximate functions that describe the evolution of the population variables over time. Effectively parameterising the population variables by t.

What if we want to consider different initial states? The problem with this approach is we cannot easily use the information we develop about the model for a different initial state – without making some huge and impractical assumptions. Therefore, we are left with no flexibility to change the initial states.

That is why this method would not scale well. If we want to generate a function for a new initialisation we have to do the same amount of work as the functions for previous initialisations. Why? Well, if we generate a function that accurately describes the trajectory of rabbit and fox populations given an initial state of $R_{Pop,0} = 70, F_{Pop,0} = 50$ – never mind how precarious that function be – how do you leverage this hard work to then understand the initial state $R_{Pop,0} = 25, F_{Pop,0} = 200$.

The Problem The fundamental problems around which we create and use differential equations is in finding trajectories of a set of parameter values over some independent variable. Rabbit and fox population levels over time is one example. We can rewrite these problems more generally as follows.

At its most basic we have a set of variables, $S_\theta = \{\theta_0, ..., \theta_{n-1}, \theta_n\}$. We use the term parameters interchangeably with variables, and we may also refer to the set of variables as a parameter space. We also have a independent variable, let's call it $t \in [0, T]$.

In most applications of differential equations the change variable is time. This is natural since everyone wants to make predictions in time. Time is not the only possible independent variable. Interestingly, the independent variable for ordinary differential equation networks is not time. Despite this, we follow convention and use $t \in [0, T]$ to represent the independent variable, to keep applied mathematicians, engineers and physicists happy.

Furthermore, an assumption for using differential equations is that we believe each parameter $\theta_i \in S_\theta$ has a real, significant, ground-truth relationship with the independent variable t. Any function that might describe this relationship will satisfy the following this structure

$$f : [0, T] \times \mathbb{R}^N \to \mathbb{R}^N f : (t, (\theta_1, ..., \theta_n)) \mapsto (\theta_1, ..., \theta_n)$$

The problem is to find an appropriate function through leveraging the rate of change of the parameters over the independent variable.

2.2 How do differential equations fit into the picture?

Defining Differential Equations A differential equation is an equation describing the relationship between functions and their derivatives. We turn to differential equations when it is easier for us to approximate how something changes rather than its actual quantity at any given moment.

For instance, we recognise that the growth of the fox (predator) population is likely to be heavily influenced by current rabbit (prey) population. For similar reasons, the growth of rabbit population is likely affected by current fox population levels. Biology experts can refine these broad assumption with evidence and so we have an empirical equation relating the rate of change of population levels with current population levels. Using this we can form a differential equation describing the relationship between current population levels (function, f) and the rate of change of population levels (derivative of the function, f' or $\frac{df}{dt}$).

In the rabbit and fox example we use expert opinion to guess the functional form of the differential equation. For low dimensional parameter spaces and when there are experts that have studied the underlying systems we can empirically guess the functional form of the differential equation and adjust it using experimentation. For high dimensional systems, or otherwise, we need different methods for finding suitable differential equations that describe a system. In such situations, we turn to function approximation.

At this point it is just important to note that we decide the differential equations. We may design the differential equation based on how we think the derivatives are influenced by the current states and other derivatives, or we may use an approximation method for the differential equation. Either way, we create this differential equation as a representation of the dynamics of the system at hand.

2.3 Objectives for Differential Equations

To solve a differential equation is to obtain the set of functions that satisfy the differential equation. Similar to how the equation given by $10 = 2x$ is satisfied by the point $x = 5$, the solution for a differential equation is a set of functions that satisfies the differential equation. In that respect the differential equation can be thought of as a set of constraints by which we can implicitly define functions.

With differential equation as the only constraint – if there are solutions – there are usually many solutions. That is why we add further constraints to obtain unique function solutions. The extra constraint we add is called the initial value problem (IVP). Given some set of functions that satisfy the differential equation – a good IVP will apply a constraint that only function will satisfy from the set. The IVP constraint is a requirement that the solution interesect a specific state at a specific point in the independent variable. We skim over certain details with this explanation.

For certain differential equations and IVPs the difficulty is we are not always able to prove the existence and uniqueness of solutions.

Our reason for wanting to find solutions to IVPs is usually one of two things.

1. **We want to obtain the trajectory for some initial state.** For example, trying to find a function that tells us how many rabbits and foxes at time $t = T$ given that for the initial time, $t = 0$, the parameter values are; $R_{pop,0} = a$ and $F_{pop,0} = b$ where a, b selected without loss of generality. We use the word trajectory in place of function at times since it nicely describes that we are interested in how the parameter values evolve over the independent variable.

2. **We want to understand the dynamics of the system better.** For example, in the rabbit and fox situation we can ask if all trajectories, or solutions, arrive at a single

destination eventually. That is, after an arbitrary amount of time do we always end up with $R_{pop,0} = a$ and $F_{pop,0} = b$, where a, b is again selected *w.l.o.g.*. This is one part of broader question of stability. There are many such questions that are worth investigating for reasons we will come back to later.

Given the challenge of finding solutions to IVPs of differential equations, we often bypass exact solution-finding and focus on obtaining approximate trajectories and building an understanding of the dynamics – dynamics refers to how the parameters change with respect to the independent variable.

We consider the challenges of obtaining or otherwise approximating trajectories for IVPs when we discuss numerical methods in Section **??**. The process of understanding dynamics is more involved. We discuss the different tools theorists and practitioners have developed to understand system dynamics across multiple later sections too. These tools include state spaces, vector fields and Jacobians amongst other things. Before diving into these topics it is useful to state the categories of differential equations.

2.4 Characteristics and Groups

2.4.1 PDEs and ODEs

Differential equations come in many forms. There are two fundamental categories of differential equations; Ordinary Differential Equations (ODEs) and Partial Differential Equations (PDEs).

There are various ways to make the distinction between ODEs and PDEs. One difference between ODEs and PDEs is in the parameter space. The parameter space for ODEs is a finite set. The parameter space for PDEs is an infinite continuum or interval. Another difference is that *ordinary* differential equations consist of functions of a single variable and their derivatives, whereas *partial* differential equations consist of functions of more than one variable and their partial derivatives.

The parameter space in the rabbit, fox population problem is the two point set given by $S_{R,F} = \{R_{Pop,t}, F_{Pop,t}\}$. Since the parameter set is discrete, the associated differential equation will be an ordinary differential equation

Now for a fresh example, suppose we have a metal rod with length L. Suppose we are interested in the heat of the rod at points $\{0, L/2, L\} \in [0, L]$ on the rod over time. Since 0 is the leftmost point of the rod, $L/2$ is halfway and L is the rightmost point, we can write the parameter space as the set $S_{Heat,Discrete} = \{Left_{Heat,t}, Middle_{Heat,t}, Right_{Heat,t}\}$.

Using theoretical guidance from physics we then pose a differential equation that dictates the rate of change of heat flow over time for each point in $S_{Heat,Discrete}$. For this example too the parameter space is a discrete set and so the associated differential equation will be an ordinary differential equation.

On the other hand, if instead, we are interested in all points on the rod, the parameter space is the continuous interval given by $S_{Heat,Continuous} = [0, L]$. The associated differential equation will be a partial differential equation. What we have described is more of a characteristic that allows us to determine whether we use partial or ordinary differential equations for a particular system.

Differential equations fit into these two categories and the study of PDEs and ODEs varies in some ways. We will see later that the parameter space for differential equation networks is a discrete set and so we focus on ODEs.

2.4.2 The Linearity and Order of Ordinary Differential Equations

Linear and Non-Linear ODEs

There is one more differential equation characteristic worth noting here. Linear and non-linear differential equations. Polynomials serve as nice metaphors for this distinction. Consider the following simple polynomial equation with one variable x and arbitrary coefficients a_i for $i \in \{0, ..., n\}$ and b

$$a_n x^n + ... + a_1 x^1 + a_0 x^0 + b = 0 \tag{2.1}$$

The degree of the polynomial is given by the size of the highest power in the polynomial, so Equation 2.1 has degree n.

Consider if instead, we replace the variable x with an arbitrary function $u(t, z)$ and we replace the power operation with a derivative with respect to t of order equal to the power. Furthermore, we replace the coefficients a_i with arbitrary functions $a_i(z)$ for $i \in \{0, ..., n\}$ and replace b with $b(z)$. Giving us,

$$a_n(z)\frac{d^n u}{dt^n} + ... + a_1(z)\frac{du}{dt} + a_0(z)u + b(z) = 0 \tag{2.2}$$

Given that Equation 2.2 consists of a function and it's derivatives we know it is a differential equation. Furthermore, all derivatives are with respect to one variable so we know this is an ordinary differential equation. Notably, Equation **??** is written as a linear combination of the function u and its derivatives – making it a linear ODE. A linear ODE is defined as an ODE that can be written as a linear combination of the function u and it's derivatives.

Order of Ordinary Differential Equations

The order of an ordinary differential equation is given by the highest derivative of the function present in the ordinary differential equation (e.g. $n = 1$ in Equation 2.2). Neural ordinary differential equations are first order ordinary differential equation. If we wish to consider higher order systems there are a few extra steps to think about. Given our needs we only focus on first order ordinary differential equations

2.5 Tools for Understanding Dynamics

2.5.1 State Space or Phase Space

Now that we have established the broad groups that ordinary differential equations fall under, we turn to a powerful tool for understanding the dynamics of systems. State space representation is a way – the way – to visualising and understanding ordinary differential equations.

For a typical low-dimensional euclidean function, like $y = 2x$, we simply plot the inputs and corresponding outputs on a graph. Through the graph we get a feel for what the function is doing. We can visually inspect important meta information or properties such as turning points and complexity.

Contrary to typical functions, ordinary differential equations are loose constraints that implicitly define sets of functions. Focusing on individual solution functions feels more comfortable because we are familiar with this approach. It is more common to work with a single function. However, we have to diverge from the typical individual function plotting approach for – at least – the following two reasons:

1. Individual functions are missing a lot of the information available from ordinary differential equations. By considering individual trajectories we are taking too microscopic a view and forego broader information about the dynamics.

2. There is typically a vast set of functions satisfying an ordinary differential equation. Obtaining the exact analytic form of these functions is challenging and often not possible. So, even if we wanted to follow this approach we are not able to do so in many situations.

At this point, let us take a step back and consider why we chose to use a first order ordinary differential equations for our problem in the first place. For the underlying problem we are interested in how the states ***evolve*** over the course of some independent variable. Therefore, we can ask – at each point in the parameter space, which direction and with what magnitude does that point move in with respect to the independent variable. Effectively we seek the derivative for each state with respect to the independent variable. The first order ordinary differential equation provides these derivatives.

Given the above, we turn to state spaces. The state space (for a first order ordinary differential equation – higher orders have slightly more complexity) is a graph for which every parameter, or degree of freedom, in our system has a unique axis, or dimension. The state space encodes information about the first order ordinary differential equation. The state space is a vector field explained in such terms in Section 2.5.2.

In the rabbit and fox population dynamics there are two parameters so the state space is given on a two dimensional graph where each point corresponds to a state in our parameter space. The x-axis represents rabbit population and the y-axis represents fox population. Since we do not have negative populations, the state space for this example is only valid in the positive regions.

Given that first order ordinary differential equations, by definition, have a finite number of parameters, we can always find a multi-dimensional graph to represent the state space. State spaces for partial differential equations differ given the continuous form parameter space. Representing higher order ordinary differential equations is more complex and discussing it is beyond the scope of this work and not entirely relevant.

2.5.2 Vector Fields

A vector field in $\mathbb{R}^n$ is defined as the assignment of an n-dimensional vector to every point in a subspace of $\mathbb{R}^n$. The state space we described in the previous section is defined on a subspace of $\mathbb{R}^n$. So we can reduce the abstraction in the definition to fit our needs by rephrasing the statement to; a vector field in $\mathbb{R}^n$, defining the state space $S \subseteq \mathbb{R}^n$, is defined as the assignment of an n-dimensional vector to every point in the state space S. Recall that the n-dimensions in the state space are because we have n parameters, or degrees of freedom, in our first order ordinary differential equation system.

The vectors we 'assign' to each point in our state space are, by and large, the derivatives of each degree of freedom with respect to the independant variable. There are more thorough

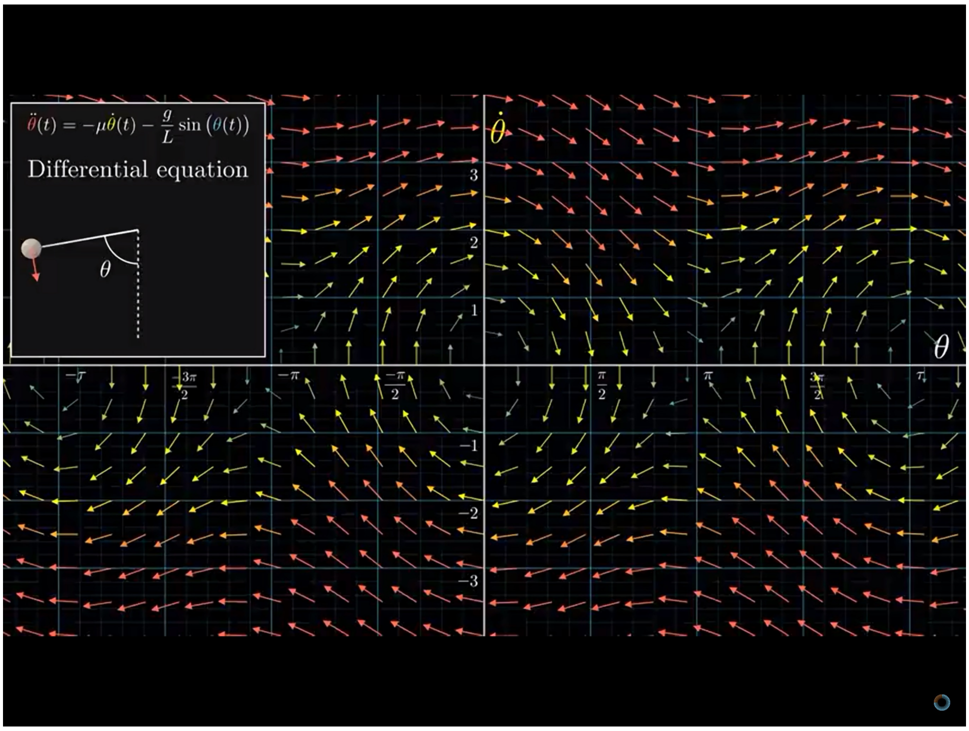

Figure 2.1: An Ordinary Differential Equation represented by a Vector Field (Sanderson (2019))

descriptions available but for neural ordinary differential equations and for understanding the basic idea of connecting first order ordinary differential equations to vector spaces the description is sufficient.

To describe this idea we focus on a simple first order ordinary differential equations with two variables given by the following equations

$$\frac{dx_1}{dt} = Ax_1 + Bx_2, \tag{2.3}$$

$$\frac{dx_2}{dt} = Cx_1 + Dx_2, \tag{2.4}$$

$$\forall\, \mathbf{x} = (x_1, x_2) \in S_{\mathbf{x}} \subseteq \mathbb{R}^2,\ A, B, C, D \in \mathbb{R}$$

We can restate the differential equation in matrix form such that we have,

$$\frac{d}{dt}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} A & B \\ C & D \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \tag{2.5}$$

which we can write in short form as,

$$\frac{d\mathbf{x}}{dt} = \mathbf{M}\mathbf{x} \tag{2.6}$$

The parameter space is a discrete set $S_{\mathbf{x}} = \{x_1, x_2\}$ with two parameters. Thus, the associated state space representation for this system has two axis, the x_1-axis and the x_2-axis. The input vector for the differential equation, given in Equation 2.6, is some position vector in the state space representation and equivalently a these position vectors in the state space representation are elements of, or points in, $S_{\mathbf{x}}$.

In terms of the definition of vector fields from before, this position vector is one of the vectors in the state space to which we assign a 2-dimensional vector. The output vector that we obtain from the differential equation is also 2-dimensional. The output vector represents the magnitude and direction of the derivative at the input position for each parameter in S_x with respect to the change parameter, t. Again, in terms of the vector field definition, the output vector is what we assign to the corresponding input vector.

Thus, the state space is a vector field to represent the derivatives for each parameter with respect to t at every position on the parameter space

These vector fields allow us to visually inspect the dynamics of the differential equations in low-dimensional systems. We may not be able to leverage visual inspection in high-dimension systems but the mathematical tools were developed through low-dimensional problems generalise well in higher dimensions and prove useful. These tools provide us with significant information about the dynamics of the system and are valuable for a wide array of use cases which we consider in more detail in later sections. Some of these tools include the Divergence, the Curl and the Jacobian. The state space representation, using vector fields also plays a crucial role in understanding stability and steady state properties in low-dimensional problems.

Now that we have established state space representations using vector fields (which we, from now on, refer to as simply 'state space representations') for first order ordinary differential equations we can think about numerical solvers. We can leverage state space representation to obtain an intuitive understanding for what it means to compute solutions to initial value problems for first order ordinary differential equations numerically.

2.6 Tools for Solving Dynamics

2.6.1 How do numerical solvers work?

As a reminder, a solution to an ordinary differential equation is a function that satisfies the ordinary differential equation. The ordinary differential equation is generally a loose constraint and so there will be a set of functions that solves the ordinary differential equation. To solve the 'initial value problem' (IVP) of an ordinary differential equation is to find a solution that satisifies a further condition given by the IVP – specifically it defines state that the solution must intersect at some value of the independent variable. Analytic methods are well studied and rigorous and provide exact solution functions. We cannot always derive the analytic form for solutions. Instead we use numerical methods to find approximations of solutions which are arbitrarily close to the analytic solutions.

Velocities in Vector Fields

A key metaphor when using and applying state spaces as vector fields is that we can think of the derivative vector as a velocity term assigned to the corresponding state – a position vector. Now, given that

$$\vec{v} \times t = \vec{d} \tag{2.7}$$

where $\vec{v} = \textit{velocity}$, $t = \textit{time}$ and $\vec{d} = \textit{displacement}$, we can use the derivative/velocity vector for some initial position vector and multiply it by a time quantity to obtain a displacement vector. We add the displacement vector to the position vector and thus form a sort of trajectory. This is the fundamental method of the numerical solver. Different numerical solvers have

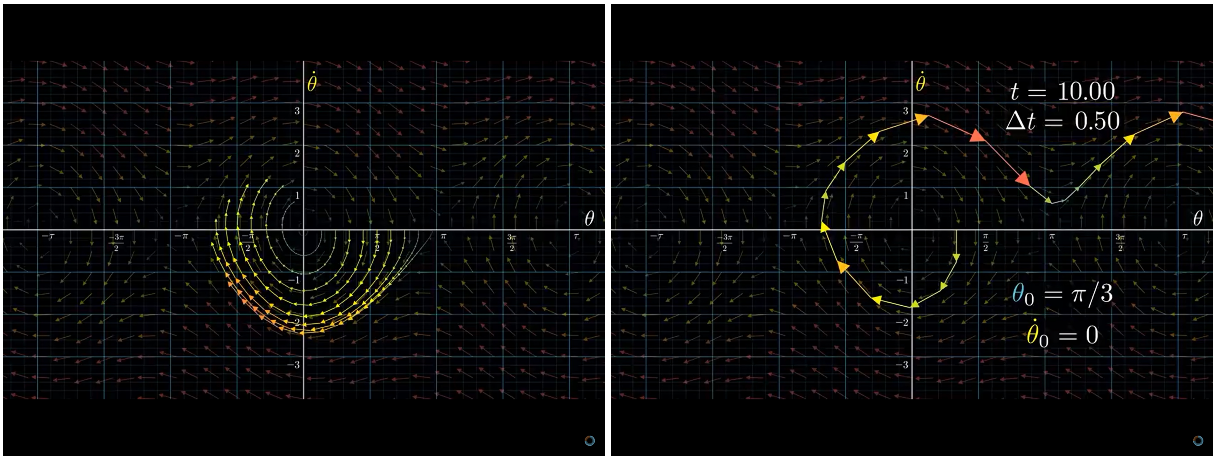

Figure 2.2: Examples of Numerical Solvers on an Ordinary Differential Equation IVP with 2 parameters represented by a Vector Field. One solver uses a steps size that is too large so the numerical solution does not accurately represent the dynamics of the system (Sanderson (2019))

different ways of choosing the time step size and they have an error calculation component – which we do not explain here. The IVP defines the initial position vector and initial time t. We compute steps, using the vector field, until we arrive at the desired final time step.

As we approach smaller step sizes for the time, the accuracy of numerical solvers increases. Rather, the error – the difference between approximate solution and analytic solution – decreases. The rate of decrease in error with respect to decrease in step size depends on the numerical solver method. As we approach infinitesimal step sizes the approximate solution approaches the analytic solution – in calculus speak.

Further Reading There are certain concepts in differential equations that are particularly relevant for Neural ODEs which we do not cover here in detail but recommend

- Divergence and Curl
- Jacobian Matrices
- Stiff and Non-Stiff ODEs

Chapter 3

Neural Ordinary Differential Equations

3.1 Motivating Neural Ordinary Differential Equations

3.1.1 Why Differential Equations and Deep Learning are meant to be together

There are compelling reasons why we might wish to combine differential equations and deep learning. These reasons come into sharper focus after we look at the broader focus of each subject.

The Focus in Machine Learning - Dynamic and Creative Function Approximators

Deep learning is a subset of machine learning. We alternatively describe machine learning as an obsession with making predictions using data. The term prediction induces the notion that we are concerned with time and the future. We generalise prediction to mean predicting outputs for unseen inputs for a system. Put in such a way, we are really just describing dynamic and advanced function approximation.

Furthermore, we are choosing a function that minimises a loss represented by the difference between the real output and the functions prediction - for some observed input point. We use observed input-output pairs and objective optimisation to improve the accuracy of our guesses for the output to unseen inputs. By way of this re-frame we can describe machine learning as the subject of function approximation using prior data.

High Calorie Data Diets By the nature of the process, having more input and output data, or examples, available for training will lead to a more accurate model. For simple input-output relationships a small amount of data will do and the value of extra examples saturates quickly. For complex systems with a high dimensionality, the models are data hungry and we need an unreasonable amount of data to train on to reach a moderate accuracy. Even after obtaining the data, the models may not be flexible enough to learn beyond a certain level.

3.1.2 The Focus in Differential Equations - If you know how it changes, you know what it is

Differential equation enthusiasts share a similar obsession with making predictions. Interestingly, the focus in study of differential equations is completely different. Where machine learning is interested in function approximation, differential equation theory is interested in extracting as much possible information from the derivatives of functions. Therefore, the study of differential equations is about one specific relationship. The relationship between functions and their derivatives.

Wide-Adoption Differential equations have found homes in a wide range of subjects. The wonderful collateral benefit of this is that the applications of differential equations have inspired developments in the abstract theory of differential equations. For instance, concepts like divergence and curl have a clear visual representation and significance in fluid dynamics but are useful in other applications of differential equations where they do not represent some obvious, visual component. Finlay et al. (2020a) uses divergence and curl in the computation of the regularisers for their proposed RNODE!

Ground-Truth Relationships There are two key assumptions we hold when we decide to apply differential equations to a system. The first assumption is that there exists some underlying meaningful derivative equation. The second assumption is that our empirical guess, theoretically founded design or approximation for the derivative equation matches or is close enough to the ground truth derivative equation. These assumptions are designed so that the theory of differential equations focuses on finding solutions for various derivative or differential equations. Notably, differential equations offer significant value in settings where we can use theoretical foundations to form the derivative equations that are similar to, or exactly match, the underlying ground truth equations.

The study of differential equations comes down to this. Given some differential equation, i.e. information that relates functions with their rates of changes – hopefully highly probable differential equations – what can we say or know about the underlying functions.

In contrast to machine learning, differential equations are not data hungry. That is, differential equations do not explicitly need pools of training data to obtain a highly probable function approximation for a system. If the differential equation is a perfect representation of the dynamics of the system then we do not need any data – or just one data point – to obtain a highly accurate function. Hopefully this provides a taste of the complimentary nature of differential equations and neural networks.

In what follows, we will explicitly describe this connection for the neural ordinary differential equation models that we are interested in. The specific models we dissect and discuss are the 'Continuous-Depth Residual Network', and the 'Continuous Normalising Flow' from Chen et al. (2018). In this section we focus on continuous depth residual networks and simply call them Neural Ordinary Differential Equations (NODEs). In a later section we consider continuous normalising flows and generative modelling.

A Differential Equations Perspective In Dupont, Doucet and Teh (2019), they describe the NODE as a 'continuous equivalent of Residual Networks'. While it is completely valid to think of NODEs as derivatives of a familiar neural network, we feel there is significant advantage in introducing and perceiving NODEs as extensions of differential equations instead. Therefore, we construct NODEs through considering the component parts of differential equations. Therefore, we turn to the state space of a NODE.

3.1.3 The Big Picture

Suppose we wish to approximate a target function with n input and output dimensions and we have a pool of training data. Typically in deep learning we use a neural network, with an appropriate architecture, as the function approximator.

With NODEs we instead use a neural network as an approximator for the ordinary differential equation of the target function. To evaluate some input point we simply 'integrate', or numerically solve, the ordinary differential equation to obtain the corresponding output. The training data is used to train the neural network to better represent or better approximate the ordinary differential equation for the target function.

Notably, with NODEs the neural network is **not** used to approximate the target function, instead it approximates the first order ordinary differential equation of some solution function that we hope is close to the target function. By improving the approximation of the differential equation we reduce the gap between the solution functions and the target function.

We noted in passing that the independant variable for differential equations is typically time, i.e. we are interested in the change in states over time, and that for NODEs this is different. Well, explicitly, with NODEs we are interested in how states change over network depth rather than time. That is, given some input, how does this input evolve as it passes through the hidden layers.

3.1.4 The Implicit Layers Paradigm

We can generalise the concept of neural ordinary differential equations further with the following re-frame, as introduced in Duvenaud, Kolter and Johnson (2021). Suppose we wish to define the following example set, $\{2, 3, 4, 5\}$. There are two methods, relevant to us, which we can use - an explicit, and implicit description.

- **Defining the Set Explicitly:** $S = \{2, 3, 4, 5\}$
- **Defining the Set Implicitly:** $S = \{x | x > 1, x < 6, x \in \mathbb{N}\}$

The ultra basic example above encapsulates the main difference between how we define a typical neural network (explicitly) and how we define a neural ordinary differential equation (implicitly). We can loosely present the idea in the following way by portraying how we might define the function approximation;

- **Explicitly:** $f(x) = NeuralNetwork(x, \theta)$, or
- **Implicitly:** $f(x) = \{f(x, t = T) | \frac{df(\cdot, t)}{dt} = NeuralNetwork(\cdot, t, \theta), f(x, t = 0) = x \ (IVP)\}$

where θ is an arbitrary parameter set that minimises the loss of the function in each case and $t \in [0, T]$ represents network depth.

Working implicitly is different to working explicitly. Each method has different advantages and limitations.

For instance, the problem of finding solutions to implicit equations is a big challenge with computationally expensive workarounds. We typically turn to numerical solvers which require a high number of Jacobian evaluations just to find the output for a single input.

On the other hand, consider the example of defining the simple set of numbers from earlier. We need to store 5 numbers if we wish to define the set explicitly, and only 3 conditions using the implicit method. Suppose that instead we wish to save the set given by $S = \{x|x > 1, x < 10^5, x \in \mathbb{R}\}$, using an explicit representation. Storing conditions typically requires less memory and so where modern flexible explicit neural networks require huge numbers of parameters, implicit methods may have memory advantages.

As we describe in a later section, the research questions stem from the opportunities that arise from thinking in implicit terms over explicit terms. Broader advantages and limitations of the implicit approach and neural ordinary differential equations are described in Chen et al. (2018), Duvenaud, Kolter and Johnson (2021) and other NODE papers.

3.1.5 Comparison to ResNets

To motivate the construction of, and propagation through a general neural ordinary differential equation, we describe the comparison with residual networks, He et al. (2016). Recall from the previous section that the difference between an explicit and implicit representation of a function is that for the former we define the function, and for the latter we describe a set of constraints – given by an ordinary differential equation and IVP – that the function obeys allowing us to identify the function uniquely.

In some cases we can describe a function both explicitly and implicitly. The ResNet is an example of this. ResNets are explicit functions which induce a set of easily obtainable constraints that allow them to be defined implicitly. That is to say we can easily construct constraints, given by a differential equation, that returns, or defines, the ResNet.

Moving a ResNet from explicit to implicit requires finding the appropriate differential equation. An n-layer ResNet defines the differential equation at n points in time/depth, given by the parameters at each layer of the explicit neural network. However, the differential equation needs to be defined at every time/depth. We believe there is no systematically accurate way to define the differential equation at points in depth where it is not defined by the explicit ResNet. Furthermore, if the differential equation is defined using some function, which it typically is we would need to solve a large system of linear equations. Obtaining the explicit representation of ResNet from an implicit definition is easier because the implicit representation stores parameter values at every depth.

Since neural ordinary differential equations are not explicit functions, the way we construct, propagate through and optimise these models differs to typical deep learning approaches significantly. Notably, there are certain requirements we need to satisfy for NODEs that are not important in explicit methods. We discuss these requirements and provide a detailed summary on the important parts in constructing NODEs. We review the process by which a NODE propagates input points and the backpropogation equivalent in NODEs.

3.2 Constructing and Propagating through NODEs

3.2.1 The Ordinary Differential Equation Network

To build a neural ordinary differential equation we need a network that specifies the ordinary differential equation. We refer to this as the ordinary differential equation network (ODENet) since it usually has many matrix multiplications which we can present using network type structures. There are certain properties that the ODENet must satisfy for specific reasons. For instance, to ensure the existence and uniqueness of solutions for initial value problems with the ODENet, the ODENet must satisfy certain continuity requirements. We provide some high level details regarding these requirements and why we need each individual one.

Existence and Uniqueness

Solving an initial value problem (IVP) on an ODE is about finding the functions that satisfy the constraints of the ODE and IVP. In neural ordinary differential equations, solving the IVP returns the path of some point through the independent variable. Most importantly this is the path a latent point travels on over the network depth to *arrive* to the corresponding output point. The solution to an IVP on an ODENet can also mean finding the path of an output point backwards through the network to *arrive* to the corresponding latent point.

Naturally, it is important that such paths exist otherwise it means the network could not evaluate certain points. Therefore we require that IVPs for the ODENet satisfy an existence condition. Furthermore, the solution of the IVP for our ODENet informs us of – for example – the output point of the network for some input latent point. If there is more than one function that solves the IVP it means there is more than one output point for some input point. This would not make sense for a neural network so we require that the IVPs for the ODENet satisfy a uniqueness condition too. In short:

- **Existence:** So every input has an output
- **Uniqueness:** So every input has no more than one output

Autonomous Ordinary Differential Equations

An autonomous ordinary differential equation is defined as an ODE that does not explicitly include the independent variable. Time is typically the independent variable for ODEs, and so autonomous systems are also referred to as time invariant. For instance we can model the effect of gravity on the trajectory of an object in the air using a differential equation. At all points in time the effect of gravity remains the same – it is a time invariant system. For a neural ordinary differential equation, where the independent variable is depth, an autonomous (or depth invariant) equation limits the expressiveness of the underlying function approximation.

The State Space

The parameter space for a simple NODE is the input space of the target function. The size of the parameter space is the flattened input dimension. We introduced and described the concept of state space representation in section 2.5.1.

The vector field for some point, z, in the state space is given by evaluating the differential equation (which is a neural network) at the input point. The differential equation returns the tangent vector at z. Evidently, we can define a vector field without evaluating the vector field at every given point.

3.2.2 Passes through ODE Networks

A powerful advantage of defining functions implicitly, and particularly through neural ordinary differential equations, is that we get more flexibility in terms of inputs and outputs. Evaluating an explicitly defined ResNet on some input is likened to solving the initial value problem, starting at the input time/depth, for an implicitly defined NODE and returning the state at the final time/depth.

By altering the start and end time/depth for an initial value problem we can use what we know about the dynamics to find the input states for output states. More generally we can change the start and end time/depths arbitrarily, within some range, for initial value problems with the ODENet. We solve initial value problems for ODENets numerically using differential equation solvers. These are commonly called 'odeint' functions in various programming packages as integrating an ODE for some specified initial value is equivalent to solving that initial value problem for the ODE.

3.2.3 ODESolve

When it comes to finding the solution function and ultimately the output point of the differential equation for any input point, z, we use some numerical method. For simplicity we consider a generic approach.

Note that, because in most applications of differential equations the independent variable is time we use, $t \in [0, T]$ to represent the independent variable for our situation, which is network depth. $t = 0$ means depth 0, and some state, $x_{t=0}$, is an input point, $x_{t=T}$, is an output point and so on.

Evaluating the Input Point z The numerical method takes in the initial point, $\vec{z}_{t_0=0}$, a position vector in the state space. It then evaluates the derivative vector for that point using the ordinary differential equation network, $\vec{v_0} = \frac{d}{dt}\vec{z}_{t_0} = ODENet(\vec{z}_{t_0})$. It then chooses the size of the t step to take depending on some method dependent error calculation and if the step size is not already fixed. After choosing $t_{step1} = t_1 - t_0 = step\ size$, it calculates the $displacement = \vec{d_0} = t_{step1} \times \vec{v_0}$. Using this the numerical method computes $\vec{z}_{t_1} = \vec{d_0} + \vec{z}_{t_0}$. Note, we start at $t = 0$ and wish to obtain the state at $t = T$, the output 'time'/depth. So we repeat the steps until we reach $\vec{z}_{t=T}$.

3.3 Continuous Normalising Flows

3.3.1 Generative Modelling: Overview of Methods

Generative modelling is the study of capturing and understanding data distributions. With generative modelling we ask what the likelihoods are of points in the data space – based on the unknown underlying distribution for the data. Typically, in real world tasks, the data is complicated, and thus the underlying distribution cannot be represented by a density function.

Algorithm 1 Baseline Procedure for Numerically Evaluating an ODENet on Input, $\vec{z}$

Given some initial point $\vec{z}_{t_0=0}$, a position vector in the state space. We wish to find $\vec{z}_{t=T}$
Let i=0
while t<T **do**
  $\vec{v}_i \leftarrow \frac{d}{dt}\vec{z}_{t_i} = ODENet(\vec{z}_{t_i})$
  Assuming a fixed step solver:
  $\Delta t_i \leftarrow$ *fixed step size*
  $\vec{d}_i = \Delta t_i \times \vec{v}_i$
  $\vec{z}_{t_{i+1}} = \vec{d}_i + \vec{z}_{t_i}$
  Prepare for next loop:
  $t_{i+1} = t_i + \Delta t_i$
  $i \leftarrow i + 1$
end while
The output is given by $\vec{z}_{t=T}$

Given this issue, the widely adopted approach for generative modelling is about creating a transformation from a simple base distribution to the complex data distribution. Notably, the objective is for the transformation is to map highly likely points in the base distribution to – what we hope are – highly likely points in the data distribution.

Given the variety of transformations available to us, and for their individual properties, generative modelling tasks branch out and focus on different aspects of the overall goal.

Maximum Likelihood Training

Taking a brief side step, the likelihood represents some information about a point in a probability distribution. Suppose we have a simple base distribution $\mathcal{A} \subseteq \mathbb{R}^n$, an unknown data distribution $\mathcal{B} \subseteq \mathbb{R}^m$, and function $f : \mathcal{A} \to \mathcal{B}$. For reasons beyond the scope of this work, we cannot naively assume the likelihood of the point $a \in \mathcal{A}$ is equivalent to the likelihood of the point $f(a) = b \in \mathcal{B}$. Furthermore, understanding the difference in the likelihood of a point in the domain to the likelihood of its corresponding point in the co-domain requires placing restrictions on the function and extra computation – we discuss this later.

To bypass this problem and still provide value, one task that generative models will focus on is finding high likelihood points in the data distribution without obtaining their exact likelihood. That is to say the function is optimised to find points in the co-domain (data distribution/space) that have high likelihoods with respect to the data distribution. This is known as sampling, and sampling based generative models outperform likelihood-based generative models in visual sample quality – for image generating tasks.

3.3.2 Change of Variables Formula

The change of variables formula provides a way for us to measure the change in probability density under a transformation. The change of variables formula – as described in the wikipedia page Probability density function (2021), and posed in detail in Jay L. Devore (2012) – is given formally in the following theorem

Definition 3.1 (Change of Variables Formula). Suppose $\mathbf{x}$ is an n-dimensional random variable with joint density f. If $\mathbf{y} = H(\mathbf{x})$, where H is a bijective, differentiable function, then $\mathbf{y}$ has density g given by

$$g(\mathbf{y}) = f\left(H^{-1}(\mathbf{y})\right)\left|det\left[\frac{dH^{-1}(\mathbf{z})}{d\mathbf{z}}\bigg|_{\mathbf{z}=\mathbf{y}}\right]\right| \tag{3.1}$$

with the differential regarded as the Jacobian of the inverse of $H(\cdot)$, evaluated at $\mathbf{y}$.

If we have a function that satisfies the conditions of the CVF in Theorem 3.1 we can use it to calculate likelihoods for samples in the unknown target data distribution. The function must be bijective and differentiable which is limiting.

In high dimensional problems, computing the determinant of the Jacobian matrix is a huge bottleneck typically with cubic cost on the number of dimension in the data space. Thus methods that use the CVF are limited further by choosing functions for which we can tractably compute the determinant of the Jacobian matrix.

Instantaneous Change of Variables

Chen et al. (2018) provide what they call the instantaneous change of variables formula (ICVF). For a differential equation network the ICVF shows that change in log probability is given by a differential equation with a trace computation on the Jacobian. The ICVF requires that the differential equation satisfies the uniqueness property. This is a weaker – and easier to satisfy – constraint than enforcing bijectivity on the explicit function. Furthermore, computing the trace of a high dimensional matrix is generally computationally cheaper than computing its determinant.

Network Structures

Dockhorn (2019) provides detailed descriptions and visual examples of the common network structures used by Grathwohl et al. (2018).

Chapter 4

Research Ideas

4.1 Combining Adversarial and Likelihood Training

In Grover, Dhar and Ermon (2017), they show that in some image generating tasks, generative adversarial networks (GANs) produce realistic looking images but have poor corresponding negative log likelihoods. Notably, they used normalising flows as the generators in the GANs for the experiments. Given the reversibility of normalising flows, they were able to calculate the negative log likelihoods.

Grover, Dhar and Ermon (2017) also found that those normalising flows trained using maximum likelihood produced less realistic samples with significantly improved corresponding negative log likelihoods. Given the interesting result, they trained a model with a hybrid approach. That is, they mixed likelihood and adversarial training. The resulting hybrid model produced better quality samples than the likelihood trained model and better negative log likelihoods than the adversarially trained model.

Normalising flows are advantageous to other reversible architectures as they allow us to generate samples quickly and tractably calculate exact negative log likelihoods. The reversibility component lends itself to some exciting possibilities too. These advantages come at the cost of restricting the architecture.

There is research advancing the normalising flow-type model in unsupervised generative modelling for images. Kingma and Dhariwal (2018) proposed the Glow model, a normalising flow with 1x1 convolutions, and conducted various unsupervised generative modelling experiments on images. To the best of our knowledge, the Glow produces the most realistic images by a normalising flow model on the CelebHQ data set Karras et al. (2018). Despite the realism of the images generated by the Glow model, the synthetic samples are easily distinguishable from real images. We believe the restricted nature of the normalising flow architecture puts a ceiling on how well these models can perform on unsupervised generative tasks.

The concept of mixing training can also be applied to continuous normalising flows in a similar manner since continuous normalising flows are reversible. Furthermore, the continuous normalising flow can have more freedom in its architecture than the normalising flow for reasons highlighted in Section 3.3.2

Research Focus Given the above, we are interesting in seeing if the added flexibility results in improved sample quality. Furthermore, given that the mixed training model from Grover, Dhar and Ermon (2017) produces more realistic samples we particularly investigate this approach, by replacing the normalising flow with a continuous normalising flow.

4.2 Latent Space Interpolation

Kingma and Dhariwal (2018) – the Glow paper – well documents the benefits of the type of reversibility that we get with normalising flows. Naturally these hold true for continuous normalising flows. An interesting task we can try, given reversible architectures, is the interpolation of points.

Interpolation of Samples We start with two samples, either generated or real, labelled x_0, x_1. Given a trained reversible architecture, we can run these samples backwards through the model to obtain corresponding latent points, z_0, z_1. We can produce n intermediate points within the latent space by calculating weighted averages of z_0, z_1 which gives the set $S = \{z_{\frac{1}{n+1}}, z_{\frac{2}{n+1}}, ..., z_{\frac{n}{n+1}}\}$. The weights used for each intermediate point are set so that they gradually move from being similar to z_0 to z_1. Each point in the set S is run forward through the model. The resulting output is the interpolated samples between x_0 and x_1. The exact method used of interpolating may vary.

In Kingma and Dhariwal (2018) they present interpolations given by their models between real data from CelebHQ. It is impressive that the model has full support on the target distribution, however, the interpolated samples appear as simply 'cut and mixes' of the two points.

Given the added flexibility with a CNF over a normalising flow, and the fact that CNFs use an implicit approach provided by a differential equation, we are interested in how well the CNF-GAN can interpolate samples.

Chapter 5

Experiments: Implementation and Results

In this chapter we walk through the experiments we conducted to investigate the research questions outlined in Chapter 4. We discuss the data, the software, the setup of the experiments, and the important decisions along the way.

Pytorch and TorchDiffEq For our experiments we decide to use the Pytorch and TorchDiffEq libraries. Many important papers have code implementations using these libraries. TorchDiffEq is an ODE numerical solver suite with full GPU support and a range of solvers, built specifically for NODEs.

5.1 CNF-GAN

Introduction

Broader Purpose Our first research question is to understand the potential flexibility, and modelling capability of continuous normalising flows. Can the CNF be trained adversarially and through likelihood to generate more visually realistic samples than counterpart normalising flow models like those found in Grover, Dhar and Ermon (2017). The process of building an experiment to research this involves deciding a model that – if stable in training and representative of the best possible for the architecture – will allow us to compare the sample quality of our model against those in Grover, Dhar and Ermon (2017) and other normalising flow models. The normalising flow found in Grover, Dhar and Ermon (2017) is most suitable to compare against since it uses the same hybrid training – adversarial and likelihood-based – that we wish to implement and try for the CNF.

Why the GAN? Grover, Dhar and Ermon (2017) provide reason to believe the adversarial and likelihood training methods each provide special advantages in unsupervised image generative tasks. They suggest that the maximum likelihood training produced far better log likelihoods – over the adversarially trained models with better samples – because likelihood training promotes 'mode-covering'. Given that we hope to push the CNF model architecture to its limits, we focus on a combined CNF-GAN type model that uses both types of learning.

Breakdown Given the profile of the experiment we need to design CNF-based models that we consider the most suitable for unsupervised image tasks. Our breakdown of the aspects of any, and every, machine learning model has five parts. The five parts are the data, the functional form (model architecture), loss function, optimisation objectives, and evaluation metrics. These are the broad groups through which we can partition most decisions.

Main Roadblock We provide a detailed critical evaluation of the following experiments in Chapter 6. One thing worth noting preemptively is that by far the biggest obstacle in progressing with these experiments was not knowing if the models collapsed because of poor modelling decisions or a lack of hyperparameter tuning.

5.1.1 Data

We focus on the MNIST handwritten digits data set for our initial experiment. Each image, or digit, in the data set has 28x28 pixels. The images are black and white, where the white is like the ink, drawing the digit, and the background is black

There are implicit levels of difficulty for unsupervised generative tasks. The levels are distinguishable on mainly the complexity and dimensionality of the underlying distribution of the data set. In some regards the MNIST data set – for generative modelling – is mid-range difficulty if we compare it with a dataset sampled from a Gaussian (simple density estimation) as a simple task and a dataset such as the CELEBHQ an extremely challenging task. The in-the-middle difficulty of MNIST contributed to why we focused on it.

Typically, conducting simple experiments first is used to inform more challenging problems. However, we are under the impression that due to the jump in complexity between data sampled from simple distributions and the MNIST data set, the insights from training a model on density estimation may not serve much purpose in the MNIST task.

Lastly, in Grathwohl et al. (2018), much like in other CNF papers, they conduct experiments on the MNIST data set. Grover, Dhar and Ermon (2017) also use the MNIST data set in their experiments. In fact, they found that on the MNIST data set, the hybrid training objective outperformed the sole adversarial and sole likelihood based training in terms of synthetic sample quality and negative log likelihoods. In other words, the wide adoption of MNIST experiments in various unsupervised generative research is motivation to focus on it.

The MNIST data set also has neat classes – one for each of the 10 digits. Therefore we can run interesting experiments in the latent space at a later stage.

5.1.2 Architecture

This part of designing the model is the most important and involved. Although from an outside machine learning perspective it might seem like deciding the architecture $\iff$ deciding the machine learning model. The optimisation objective, loss functions, and evaluation metrics are distinct to the architecture but significantly important too.

We propose a CNF-GAN architecture which means we need to define the CNF and GAN elements of the model. The data dimensionality dictates the input and output size of the model. We focus on loss functions (Sec. 5.1.3) and optimisation objectives (Sec. 5.1.4) separately despite being interlaced decisions with architecture.

High-Level Our vision for the architecture was a generative adversarial network where the generator is set to be some continuous normalising flow and the discriminator from a DCGAN-type network. We note that the difficulty of our task was **not** in producing these models. The goal of our research was not to recreate a better CNF or GAN individually but to combine two already implemented such models. The process of building our own individual models from scratch would have taken away time from researching the key question of combining learning types.

There are many DCGANs available for training on MNIST data with full GPU support on pytorch. Furthermore, Grathwohl et al. (2018) and Finlay et al. (2020b) provide code for continuous normalising flows for the MNIST data set, also with full GPU support through pytorch.

CNF Generator- FFJORD In our MNIST models we mainly tried two implementations of the CNF, both rooted in Grathwohl et al. (2018). Our first FFJORD-based CNF forked the github repo given by Grathwohl et al. (2018). The generator model is an 'ODENVP' object. The ODENVP is simply multiple stacked CNFs. Each stacked CNF contains a chain of modules. The modules are either ODE functions or squeeze/unsqueeze layers. The ODE function defines an ODE network composed of special network structures and activation functions. An expanding discussion of these ODE network layers is given by Dockhorn (2019). The squeeze and unsqueeze layers are explained in detail in Dinh, Sohl-Dickstein and Bengio (2016).

DCGAN Discriminator We forked a basic DCGAN discriminator network – with MNIST training functionality, and pytorch GPU Support – from Kadel (2019). The DCGAN architecture follows the model suggested in Radford, Metz and Chintala (2016).

Hyperparameters and Decisions We list a few of the important decisions that needed to be made. Some of these were hyperparameters, for others we made theoretically informed choices.

Continuous Normalising Flow

1. Number of stacked CNF blocks
2. Number of CNFs within each block
3. Dimension and number of intermediary layers
4. Zero-ing the last layer
5. ODENet hyperparameters
    (a) Activation functions
    (b) Layer types
    (c) Layer configurations; kernel sizes, strides and padding
    (d) Logit Transformation (alpha parameter)

(e) Regularisation hyperparameters - for RNODE
   i. Kinetic energy
   ii. Jacobian norm two
   iii. Directional penalty
   iv. Total derivative
6. Numerical solver hyperparameters (test/train)
   (a) Solver type
   (b) Relative tolerance (rtol)
   (c) Absolute tolerance (atol)
   (d) Step size (for fixed step size solvers)
   (e) First step (for adaptive solvers)
   (f) Time/Depth Length (Depth)

DCGAN

1. Convolutional layers
2. Batch normalisation
3. Activation functions

5.1.3 Loss Function

The discriminator and generator loss are different. For the discriminator we use a basic adversarial loss based calculated using binary cross entropy.

Generator (CNF) Loss There are a variety of ways we could have posed the loss function since there are two losses – the maximum likelihood loss and adversarial loss. Empirically, it seems that both losses would not oppose each other. Since the training style of each method is quite different, it is possible that at certain points in training both losses would oppose each other – resulting in a lower combined effectiveness.

The way we configure the loss function is by adding the maximum likelihood and adversarial loss terms. The maximum likelihood loss term is given by the bits per dimension of the sample. The bits per dimension is calculating by taking a sample, running it back through the model to obtain a corresponding latent state. Obtaining the likelihood of that point based on the latent space distribution. Propagating the likelihood forward to obtain the likelihood of the sample in the complex underlying data distribution. The adversarial loss is the binary cross entropy on the output of the discriminator on the samples created by the generator.

5.1.4 Optimisation Objective

For both the generator and discriminator we use the Adam optimiser, Kingma and Ba (2015). We train with a batch size of 200. We set the learning rate for the generator CNF between

$1 \times e^{-3}$ to $1 \times e^{-5}$. This is similar to the discriminator, with the discriminator always taking a slightly higher learning rate. We vary the maximum gradient normalisation which clips gradients beyond a limit – only used in the CNF.

5.1.5 Hyperparameters

There are two types of hyperparameters for models with stability issues. The hyperparameters that are crucial to the stability properties of a model and those that are not. Rather than a grouping, it is more of labelling which helps us focus on a smaller set of hyperparameters for stabilising dynamics and a larger set later for optimising performance. Often we do not know for sure which hyperparameters strongly affect stability and certain other model failure issues. With practice machine learning experts pick up on what needs to be adjusted based on the reason for collapse – based on experience and reading.

The importance of minimising the number of hyperparameters cannot be understated. Training time is simply too long.

5.1.6 Timeline

The first three weeks were exclusively spent understanding the premise of neural ordinary differential equations and a building a broad picture of deep learning. We continued to build our understanding of NODEs through the course of the project. We spent the following two weeks deciding which library to build our model on top of and which code base to fork from. There are multiple code bases on NODEs, we needed one that was reliable, up to date and easily adaptable. We chose to fork from the FFJORD by Grathwohl et al. (2018) – implemented on pytorch. We spent the following four weeks creating our initial CNF-GAN, debugging the mode and learning how to use HEX – the university provided GPU cloud. In August we hoped to perform a full hyperparameter search on the CNF-GAN model we created. Given certain dilemmas, as described in Section 6.2, we instead spent August building variations of the CNF-GAN.

5.1.7 Results

All of our models either did not remain stable or the loss exploded. Despite this we were unable to disprove that the models are flexible enough to capture the underlying MNIST data set. Proving the hypothesis required a model to work, to disprove would require theoretical justification. Alternatively, using experimentation we could have provided strong reasons against the hypothesis – through running a hyperparameter search on a representative CNF-GAN.

For reasons we provide in Section 6.2, we did not run a full hyperparameter search for a model. We instead ran different models with generally sound hyperparameter configurations. We provide some results from these runs before they collapse (see Figure 5.1.

5.2 Training Variations

Regularised Neural Ordinary Differential Equation

One variation to our original model was the CNF-GAN were the FFJORD CNF is replaced by a RNODE proposed in Finlay et al. (2020a). We use a DCGAN for the discriminator here

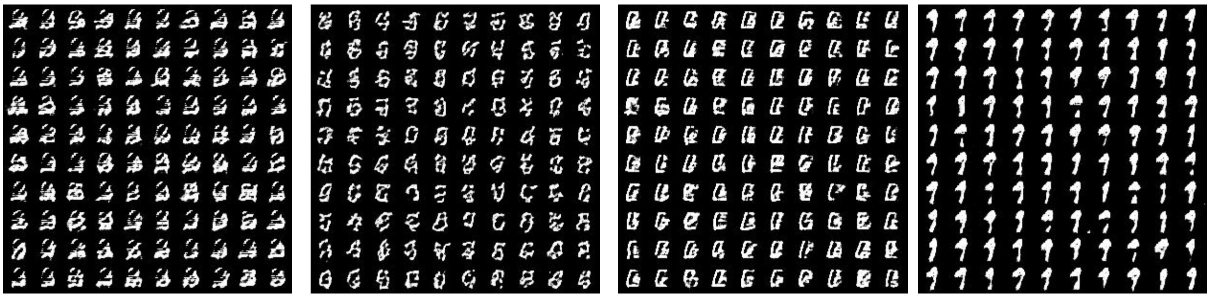

Figure 5.1: Synthetic Samples Generated by the CNF-GAN (FFJORD-DCGAN) within 4 Epochs

Example Learning Metrics of the CNF-GAN (FFJORD-DCGAN) on the MNIST dataset

Itr	Step	D_loss	G_loss	D(x)	D(G(x1))	L.Lik Loss	L.Lik + Adv Loss
0	0	1.500	0.942	0.482	0.526	19.674	19.674
1	10	0.797	0.775	0.850	0.468	3.990	16.703
2	20	0.613	0.973	0.890	0.389	5.112	13.247
3	30	0.416	1.553	0.861	0.229	2.504	10.487
4	40	0.304	2.460	0.825	0.093	2.672	8.372
5	50	0.924	0.716	0.884	0.521	2.203	6.756
6	60	1.698	1.725	0.302	0.302	2.288	5.558
7	70	1.502	0.835	0.511	0.523	2.359	4.716
8	80	1.519	0.943	0.527	0.501	2.168	4.061
9	90	1.443	0.880	0.513	0.485	2.072	3.589
10	100	2.247	0.321	0.600	0.793	2.041	3.197

too. We experienced similar issues to the FFJORD implementation. In Figure 5.2 we present samples from a training run that collapsed because of exploding gradients.

U-Net based Discriminator

We also implemented a CNF-GAN that uses an RNODE for the generator and a U-net discriminator for the discriminator. This experiment distracted the main objective of the project significantly in the end. In defense, the U-net shows high potential for improving stability. Considering that the issue we predominantly faced with our implementations were stability problems we felt it was worthwhile to pursue this route before completing a full hyperparameter search.

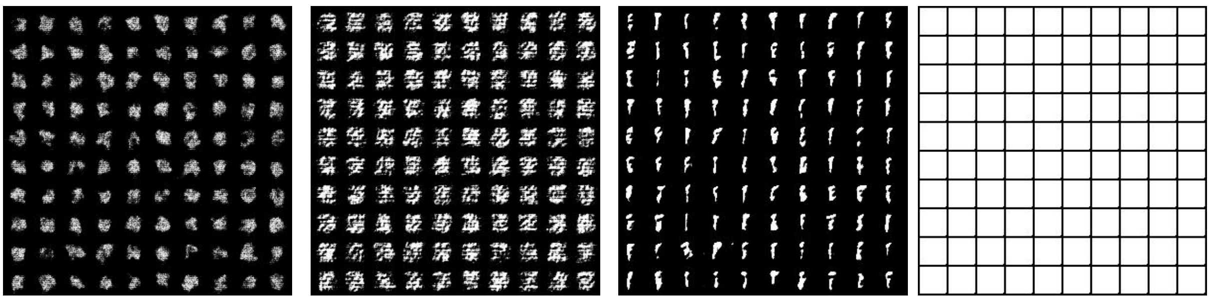

Figure 5.2: Synthetic Samples Generated by the CNF-GAN (RNODE-DCGAN) – this training run collapsed

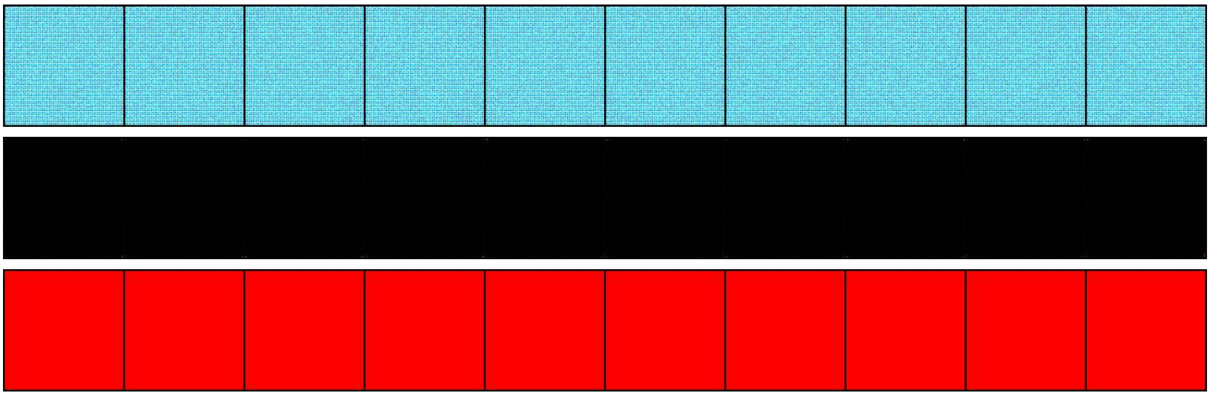

Figure 5.3: Synthetic Samples Generated by the CNF-GAN (RNODE-U-Net) on the CelebHQ data set with Jacobian Trace Computation Eliminated

The mistake we made was to build the CNF-U-net for the CelebHQ data set. The CelebHQ data set is really large and has a significantly more complex underlying distribution than MNIST. It would have been better to achieve results or prove a hypothesis on the MNIST before moving to CelebHQ – given the constraints on time.

Jacobian Trace Computation-Free Adversarial Training

To compute a loss for the generator through adversarial training we need samples from the generator. We run the samples through the discriminator and obtain a loss term based on a computation on the output of the discriminator for the synthetic samples. Notably, adversarial training only needs samples from the generator, it does not explicitly need the Jacobian trace computation that likelihood training explicitly needs – in order to calculate the log likelihoods of samples. Furthermore, the Jacobian trace computation is a big bottleneck for evaluating the CNF models.

Given that we are still developing an understanding for the concepts surrounding neural ordinary differential equations we thought the following idea – Jacobian trace computation-free sample generation – would not have issues. Specifically we thought we could eliminate the Jacobian trace computations specifically when using the model for generating samples. For reasons that we did not get a chance to fully appreciate this would not work and the samples would be degenerate without using the Jacobian trace computation.

We implemented a way to run the idea – before knowing it may not theoretically work – within the RNODE-Unet model for the CelebHQ. The model was significantly faster but the samples were degenerate. Note that we did not get a chance to test whether the samples were degenerate because of the implementation of the idea or because of the nature of the experiment. We restored the Jacobian trace computations to the same model and it also return degenerate samples. Samples from the Jacobian trace computation free model are presented in Figure 5.3

Hybrid Training

We also tried some variations on the hybrid loss function. For most of our experiments the loss for the generator is the sum of the loss from the likelihood and adversarial training. We considered varying the effect of each loss term depending on the performance of the model.

We decided that these questions come packed with complexity and could serve as research question. Therefore we left the losses as the simple addition of adversarial and likelihood loss terms.

Latent Space Interpolation

Since we did not manage to successfully train a CNF-GAN on the MNIST data set we did not get a chance to run the latent space experiments. Comparing reversible generative models through various latent space experiments is an extra tool we have for comparing and understand models.

Chapter 6

Critical Evaluation and Conclusions

6.1 Achievements

Objective: Exploring Differential Equations With this work we hoped to visit the important aspects of differential equation theory that lends itself to neural ordinary differential equations from Chen et al. (2018). The discussion of differential equations is mostly intended to motivate and provide intuition behind the conventional ways of thinking about differential equations. Since differential equations are unlike many topics in data science the explanations we provide will benefit data scientists in better understanding development in this area. The work provides a bird's-eye of what differential equations are, why we use them, and how we think about them.

Objective: CNF-GAN We originally hoped to prove by example that a continuous normalising flow based generative adversarial network (CNF-GAN) could successfully train on the MNIST data set in an unsupervised, generative capacity. The objective included a study that compared the CNF-GAN against a CNF in terms of the quality of samples generated and log likelihoods of samples. Over the course of the project we produced and implemented a CNF-GAN on the MNIST data set. We also constructed interesting variations to the CNF-GAN. We were unable to combat the stability and collapse issues faced by the model so the subsequent comparison study was not reached

Objective: Latent Space Interpolation Furthermore, we wished to compare the quality of latent space interpolation in the CNF-GAN against the normalising flow from Kingma and Dhariwal (2018) and a standalone CNF. This objective was interesting to us because, in Chen et al. (2018) they suggest that for density estimation tasks, the learning by the CNF is more interepretable and smoother than the normalising flow. We did not conduct this study because the CNF-GAN did not train successfully.

6.2 Critique of the Process

Roadblocks Each epoch in training (for MNIST) took upwards of 25 minutes. Plus, the training dynamics were quite unstable. The gradient norm and loss in bits per dimension often shot up during training. After repeatedly unstable training runs despite lowering learning rates and applying gradient clipping we questioned whether the model was doing what we intended. We were concerned about three things;

1. Is there a bug in the code that means the model is not doing what we intend, e.g. rogue control flow
2. Is the way in which we are combining and applying learning theoretically wrong
3. Is the model inherently unstable, and is it worth instead building a more stable counterpart

The first point is a natural concern for a software project and especially problematic for GPU based software where unit testing is harder to conduct and access to variables and the environment is limited. To clear this up we employed tools for debugging picked up from the past year of study. While this was probably not the root issue it consumed considerable amounts of time as it needed to be checked.

Designed to Fail? The second issue was more troubling. It is challenging to understand which decisions are theoretically sound and which decisions are not for new models. While for some choices there is literature available which analyses the decisions in depth – many choices have little definitive theory. For instance, the adversarial and likelihood losses could be completely at ends to one another and combining them might always lead to model collapse or gradient explosion. Given more experience with machine learning we may have a better understood the common root causes for certain issues. On the topic of combining loss terms, we recently found a paper which suggests a way to combine different loss terms. Applying these ideas to the CNF-GAN in future work would be interesting.

Lesson: Extensive Background Reading Ideally we should have read and studied differential equations and neural ordinary differential equations more thoroughly before undertaking this project. In the process of reading papers it is helpful to summarise key points and make note of potential future work.

Trying Different Models The question of whether the model needed to be changed drove the significant delays. In order to disprove the hypothesis we need to run a full hyperparameter search on a representative model and show that every run collapses. All runs failing is as close to knowing if the model is not flexible enough for this task. Running a full hyperparameter search on such a model; with many hyperparameters and long training times is very time consuming. When the experiments failed after a few combinations of hyperparameters – the ones we considered crucial to stability such as learning rate and gradient normalisation – we pursued different variations of the model – in case ours was not representative of the best.

Explore-Exploit We compare this to the explore-exploit dilemma. In hindsight we should have set a deadline for switching from exploring – trying different models – to exploiting – running the full hyperparameter search on our chosen model. Instead as various approaches did not immediately work we tried versions of CNF-GAN-based models in an explore frenzy. These included a FFJORD-RNODE CNF, a U-net discriminator, Jacobian free adversarial training, various loss functions and even temporarily trying the CELEBHQ dataset.

Lesson: Cut-Offs Pre-decide the explore-exploit ratio of time. The original plan included a breakdown of how long we have for each experiment, we should have instead provided a time for when we stop designing the model and experiment and start testing the model and experiment. There should be hard cut-off deadlines after which either we start doing something – like conducting a full hyperparameter search – or stop doing something – like trying new models and significant changes.

Transition from Debugging to Making Significant Changes Another interesting observation is in regards to the coding stage. When we edit code there are many different reasons. Sometimes, it is a minor, silly error such as syntax mistakes or the control flow is not running as intended. This is more than debugging, it also includes edits that do not change what you might describe with pseudo-code. At other times we make edits that are based on a change at a higher level – changing the pseudocode.

The debugging stage can be slow and time consuming, and for basic changes we do not log or note the edits for discussion. Ideally for the second kind of edit we want to log these so we have more clarity on how the model works and the process of building the model. Somewhere along the line of getting these models ready and debugging them, the 'debug' edits are switched with 'high-level' edits. Therefore they are easily missed and not noted.

Lesson: The Significance of First Drafts The process of writing a first draft is crucial for a research project. Thoughts and ideas of where the research might go are unbounded as long as they stay abstract and in the mind. By writing a first draft we ground and bound the objectives, ideas and our thoughts about the project – thus narrowing down precisely what we wish to research and importantly, what we are not researching. For this work, we wrote miscellaneously throughout the course of the project but started the first draft late – in August. We felt an subconscious barrier to writing the first draft due to a variety of reasons. For example, a strong deterrent for starting the first draft was that we believed it was important to obtain results first and so the priority on time should be to focus on finishing the code first.

We later realise this to be a crucial error of judgement for the following reason. By writing – even an incomplete – first draft early, we can better judge which directions are relevant and useful, and which directions are a distraction. If we wrote a first draft early, we may have been less likely to pursue the CelebHQ data set before finishing experiments with MNIST.

Another crucial benefit of an early first draft is that it provides a spatial view of the project and its contents. If we write a paragraph about a certain question or idea and we already have a first draft we can immediately find where it fits in the context of our project – and whether it even fits. Without a first draft it becomes significantly harder to judge where the piece of writing or idea fits into the project – or whether it is relevant or not – making the writing more burdensome than beneficial.

6.3 Future Work

Reducing Computations in Neural Ordinary Differential Equations

A significant barrier in the widespread adoption of neural ordinary differential equations and reversible generative models is the time cost involved in either sampling or training. There are different ways these costs can be reduced. A few examples are;

- The complexity of dynamics of ODENets at a later training stage – Finlay et al. (2020a) regularise the NODE to simplify dynamics. Higher complexity and erratic dynamics require more function evaluations.

- Increasing the number of accepted steps and reducing the number of rejected steps by the solver by eliminating rejections triggered by unnecessary reasons – as Dupont, Doucet and Teh (2019) does.

These examples serve as inspiration for how we can reduce computations for NODEs. In eliminating Jacobian trace computations when generating samples (see Section 5.2) we were aiming to reduce unnecessary computations. It turns out these Jacobian trace computations are likely very important, even for sample generation. It would be interesting to investigate why they are important. This may lead to discovering situations for which we could replace them and reduce computations.

Multiple Loss Functions

Dosovitskiy and Djolonga (2020) investigate an interesting way to train a model with multiple loss functions. We believe that variations of this idea can be applied in some sense to the hybrid loss of a CNF-GAN-type model. Adversarial and likelihood loss terms both present unique advantages as described in Grover, Dhar and Ermon (2017). Researching how we can maximise the synergy between the losses may be fruitful in the context of generative modelling for images.

Bibliography

Arjovsky, M., Chintala, S. and Bottou, L., 2017. Wasserstein generative adversarial networks [Online]. In: D. Precup and Y.W. Teh, eds. *Proceedings of the 34th international conference on machine learning*. PMLR, *Proceedings of Machine Learning Research*, vol. 70, pp.214–223. Available from: `http://proceedings.mlr.press/v70/arjovsky17a.html`.

Ayyubi, H.A., Yao, Y. and Divakaran, A., 2020. Progressive growing of neural {ode}s [Online]. *Iclr 2020 workshop on integration of deep neural models and differential equations*. Available from: `https://openreview.net/forum?id=vO4cDCAVFp`.

Borji, A., 2018. Pros and cons of GAN evaluation measures. *Corr* [Online], abs/1802.03446. `1802.03446`, Available from: `http://arxiv.org/abs/1802.03446`.

Brock, A., Donahue, J. and Simonyan, K., 2018. Large scale GAN training for high fidelity natural image synthesis. *Corr* [Online], abs/1809.11096. `1809.11096`, Available from: `http://arxiv.org/abs/1809.11096`.

Chen, R.T.Q., Rubanova, Y., Bettencourt, J. and Duvenaud, D.K., 2018. Neural ordinary differential equations [Online]. In: S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi and R. Garnett, eds. *Advances in neural information processing systems*. Curran Associates, Inc., vol. 31. Available from: `https://proceedings.neurips.cc/paper/2018/file/69386f6bb1dfed68692a24c8686939b9-Paper.pdf`.

Chen, X., Duan, Y., Houthooft, R., Schulman, J., Sutskever, I. and Abbeel, P., 2016. Infogan: Interpretable representation learning by information maximizing generative adversarial nets [Online]. In: D. Lee, M. Sugiyama, U. Luxburg, I. Guyon and R. Garnett, eds. *Advances in neural information processing systems*. Curran Associates, Inc., vol. 29. Available from: `https://proceedings.neurips.cc/paper/2016/file/7c9d0b1f96aebd7b5eca8c3edaa19ebb-Paper.pdf`.

Dinh, L., Sohl-Dickstein, J. and Bengio, S., 2016. Density estimation using real NVP. *Corr* [Online], abs/1605.08803. `1605.08803`, Available from: `http://arxiv.org/abs/1605.08803`.

Dockhorn, T., 2019. Generative modeling with neural ordinary differential equations.

Dosovitskiy, A. and Djolonga, J., 2020. You only train once: Loss-conditional training of deep networks [Online]. *International conference on learning representations*. Available from: `https://openreview.net/forum?id=HyxY6JHKwr`.

Dupont, E., Doucet, A. and Teh, Y.W., 2019. Augmented neural odes [Online]. In: H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox and R. Garnett, eds. *Advances in neural information processing systems*. Curran Associates, Inc., vol. 32. Available from: `https://proceedings.neurips.cc/paper/2019/file/21be9a4bd4f81549a9d1d241981cec3c-Paper.pdf`.

Duvenaud, D., Kolter, Z. and Johnson, M., 2021. Available from: `http://implicit-layers-tutorial.org/`.

Finlay, C., Jacobsen, J.H., Nurbekyan, L. and Oberman, A.M., 2020a. How to train your neural ode: the world of jacobian and kinetic regularization [Online]. *Icml*. pp.3154–3164. Available from: `http://proceedings.mlr.press/v119/finlay20a.html`.

Finlay, C., Jacobsen, J.H., Nurbekyan, L. and Oberman, A.M., 2020b. How to train your neural ode: the world of jacobian and kinetic regularization. `2002.02798`.

Glorot, X., Bordes, A. and Bengio, Y., 2011. Deep sparse rectifier neural networks [Online]. In: G. Gordon, D. Dunson and M. Dudík, eds. *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. Fort Lauderdale, FL, USA: PMLR, *Proceedings of Machine Learning Research*, vol. 15, pp.315–323. Available from: `https://proceedings.mlr.press/v15/glorot11a.html`.

Goodfellow, I.J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y., 2014. Generative adversarial nets. *Proceedings of the 27th international conference on neural information processing systems - volume 2*. MIT Press, NIPS'14, p.2672–2680.

Grathwohl, W., Chen, R.T.Q., Bettencourt, J., Sutskever, I. and Duvenaud, D., 2018. FFJORD: free-form continuous dynamics for scalable reversible generative models. *Corr* [Online], abs/1810.01367. `1810.01367`, Available from: `http://arxiv.org/abs/1810.01367`.

Grover, A., Dhar, M. and Ermon, S., 2017. Flow-gan: Bridging implicit and prescribed learning in generative models. *Corr* [Online], abs/1705.08868. `1705.08868`, Available from: `http://arxiv.org/abs/1705.08868`.

He, K., Zhang, X., Ren, S. and Sun, J., 2016. Deep residual learning for image recognition [Online]. *2016 ieee conference on computer vision and pattern recognition (cvpr)*. pp.770–778. Available from: `https://doi.org/10.1109/CVPR.2016.90`.

Hong, Y., 2019. Comparison of generative adversarial networks architectures which reduce mode collapse. *Corr* [Online], abs/1910.04636. `1910.04636`, Available from: `http://arxiv.org/abs/1910.04636`.

Igorevic, A.V. and Silverman, R.A., 1998. *Ordinary differential equations*. The MIT Press.

Jay L. Devore, K.N.B.a., 2012. *Modern mathematical statistics with applications*, Springer Texts in Statistics. 2nd ed. Springer-Verlag New York.

Kadel, A., 2019. Akashkadel/dcgan-mnist: Pytorch implementation of dcgan. Available from: `https://github.com/AKASHKADEL/dcgan-mnist`.

Karras, T., Aila, T., Laine, S. and Lehtinen, J., 2018. Progressive growing of gans for improved quality, stability, and variation. `1710.10196`.

Karras, T., Laine, S. and Aila, T., 2018. A style-based generator architecture for generative adversarial networks. *Corr* [Online], abs/1812.04948. `1812.04948`, Available from: `http://arxiv.org/abs/1812.04948`.

Kidger, P., Chen, R.T.Q. and Lyons, T.J., 2020. "hey, that's not an ode": Faster ODE adjoints with 12 lines of code. *Corr* [Online], abs/2009.09457. 2009.09457, Available from: https://arxiv.org/abs/2009.09457.

Kingma, D.P. and Ba, J., 2015. Adam: A method for stochastic optimization [Online]. In: Y. Bengio and Y. LeCun, eds. *3rd international conference on learning representations, ICLR 2015, san diego, ca, usa, may 7-9, 2015, conference track proceedings*. Available from: http://arxiv.org/abs/1412.6980.

Kingma, D.P. and Dhariwal, P., 2018. Glow: Generative flow with invertible 1x1 convolutions. 1807.03039.

Lu, Y., Zhong, A., Li, Q. and Dong, B., 2017. Beyond finite layer neural networks: Bridging deep architectures and numerical differential equations. *Corr* [Online], abs/1710.10121. 1710.10121, Available from: http://arxiv.org/abs/1710.10121.

Lucic, M., Kurach, K., Michalski, M., Gelly, S. and Bousquet, O., 2018. Are gans created equal? a large-scale study [Online]. In: S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi and R. Garnett, eds. *Advances in neural information processing systems*. Curran Associates, Inc., vol. 31. Available from: https://proceedings.neurips.cc/paper/2018/file/e46de7e1bcaaced9a54f1e9d0d2f800d-Paper.pdf.

Nash, J.F., 1950. Equilibrium points in n-person games. *Proceedings of the national academy of sciences* [Online], 36(1), pp.48–49. https://www.pnas.org/content/36/1/48.full.pdf, Available from: https://doi.org/10.1073/pnas.36.1.48.

Odena, A., 2016. Semi-supervised learning with generative adversarial networks. 1606.01583.

Probability density function, 2021. Available from: https://en.wikipedia.org/wiki/Probability_density_function#Vector_to_vector.

Qin, C., Wu, Y., Springenberg, J.T., Brock, A., Donahue, J., Lillicrap, T. and Kohli, P., 2020. Training generative adversarial networks by solving ordinary differential equations [Online]. In: H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan and H. Lin, eds. *Advances in neural information processing systems*. Curran Associates, Inc., vol. 33, pp.5599–5609. Available from: https://proceedings.neurips.cc/paper/2020/file/3c8f9a173f749710d6377d3150cf90da-Paper.pdf.

Radford, A., Metz, L. and Chintala, S., 2016. Unsupervised representation learning with deep convolutional generative adversarial networks. *Corr*, abs/1511.06434.

Ronneberger, O., Fischer, P. and Brox, T., 2015. U-net: Convolutional networks for biomedical image segmentation. *Corr* [Online], abs/1505.04597. 1505.04597, Available from: http://arxiv.org/abs/1505.04597.

Rubanova, Y., Chen, R.T.Q. and Duvenaud, D.K., 2019. Latent ordinary differential equations for irregularly-sampled time series [Online]. In: H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox and R. Garnett, eds. *Advances in neural information processing systems*. Curran Associates, Inc., vol. 32. Available from: https://proceedings.neurips.cc/paper/2019/file/42a6845a557bef704ad8ac9cb4461d43-Paper.pdf.

Ruthotto, L. and Haber, E., 2021. An introduction to deep generative modeling. 2103.05180.

Salimans, T., Goodfellow, I.J., Zaremba, W., Cheung, V., Radford, A. and Chen, X., 2016.

Improved techniques for training gans. *Corr* [Online], abs/1606.03498. 1606.03498, Available from: http://arxiv.org/abs/1606.03498.

Sanderson, G., 2019. 3blue1browngithub. Available from: https://github.com/3b1b/videos/tree/master/_2019/diffyq/part1.

Schönfeld, E., Schiele, B. and Khoreva, A., 2020. A u-net based discriminator for generative adversarial networks. *Corr* [Online], abs/2002.12655. 2002.12655, Available from: https://arxiv.org/abs/2002.12655.

Theis, L., Oord, A. van den and Bethge, M., 2016. A note on the evaluation of generative models [Online]. *International conference on learning representations*. Available from: http://arxiv.org/abs/1511.01844.

Appendix A

Results

Example Learning Metrics of the CNF-GAN (FFJORD-DCGAN) on the MNIST dataset

Itr	Step	D_loss	G_loss	D(x)	D(G(x1))	L.Lik Loss	L.Lik + Adv Loss
0	0	1.500	0.942	0.482	0.526	19.674	19.674
1	10	0.797	0.775	0.850	0.468	3.990	16.703
2	20	0.613	0.973	0.890	0.389	5.112	13.247
3	30	0.416	1.553	0.861	0.229	2.504	10.487
4	40	0.304	2.460	0.825	0.093	2.672	8.372
5	50	0.924	0.716	0.884	0.521	2.203	6.756
6	60	1.698	1.725	0.302	0.302	2.288	5.558
7	70	1.502	0.835	0.511	0.523	2.359	4.716
8	80	1.519	0.943	0.527	0.501	2.168	4.061
9	90	1.443	0.880	0.513	0.485	2.072	3.589
10	100	2.247	0.321	0.600	0.793	2.041	3.197
11	110	1.178	1.246	0.533	0.351	2.022	2.913
12	120	1.854	0.502	0.675	0.664	2.179	2.715
13	130	1.806	0.637	0.492	0.630	2.056	2.560
14	140	1.500	0.701	0.487	0.499	1.981	2.423
15	150	1.622	0.756	0.444	0.512	2.048	2.325
16	160	1.428	0.914	0.459	0.441	2.014	2.243
17	170	1.528	0.675	0.519	0.559	2.009	2.178
18	180	1.649	0.709	0.441	0.528	1.939	2.115
19	190	1.657	0.637	0.480	0.568	1.834	2.053
20	200	1.997	0.511	0.426	0.663	1.871	2.006
21	210	1.364	1.016	0.419	0.361	2.002	1.980
22	220	1.832	0.430	0.543	0.686	1.855	1.945
23	230	1.687	0.937	0.361	0.447	1.807	1.911
24	240	1.482	0.865	0.441	0.459	1.907	1.891
25	250	1.749	0.446	0.560	0.662	1.789	1.871
99	90	0.907	1.247	0.594	0.307	2.660	1.774